Fattorioni e semifattorioni

Working paper / Nota didattica / versione 31 marzo 2026

Sergio L Invernizzi

Società dei Naturalisti e Matematici di Modena, via Università 4, 41121 Modena
Centro Ricerche Didattiche U. Morin, via S. Giacomo 4, 31017 Pieve del Grappa (TV)
e–mail: `invernizzi2@gmail.com`

2.60 The loneliness of the factorions. Two tiny factorions are 1 = 1! and 2 = 2! The largest known factorion is 40,585; it can be written as $40,585 = 4! + 0! + 5! + 8! + 5!$ (The factorion 40,585 was discovered in 1964 by R. Dougherty using a computer search.)[a] Various proofs have been advanced indicating that 40,585 is the largest possible factorion, and that humans will never be able to find a greater factorion.

Pickover, C. A. (2005). *A passion for mathematics: Numbers, puzzles, madness, religion, and the quest for reality* (p. 305). John Wiley & Sons.

[a] L'affermazione di Pickover secondo cui "il fattorione 40585 fu scoperto nel 1964 da R. Dougherty tramite una ricerca computerizzata" non sembra riferirsi ad un articolo pubblicato. Le uniche tracce di questa attribuzione provengono da ripetizioni incontrollate dell'affermazione di Pickover in molti post informali.

Sommario Lo scopo della nota è duplice. Per primo, studiamo alcuni numeri "curiosi", i noti fattorioni, che sono i numeri interi uguali alla somma dei fattoriali delle loro cifre in una data base, accostando a questi i semifattorioni, che definiamo come i numeri uguali alla somma dei semifattoriali delle loro cifre. Introduciamo una stima formale delle loro corrispondenti numerosità basata sulla funzione di Lambert W, e determiniamo in tal modo una limitazione del numero delle loro cifre nella base scelta. Questo riduce la ricerca ad un insieme finito, e rende il problema computabile, ad esempio tramite Python. Indichiamo poi come questo problema possa rappresentare un interessante caso paradigmatico – proponibile in talune classi – della tecnica di "riduzione al finito", che consente di risolvere importanti problemi (primo fra tutti, storicamente, il Teorema dei Quattro Colori) riportandoli con procedimenti teorici ad un numero finito di casi, che poi possono essere esaminati da un sistema automatico: le cosiddette *dimostrazioni assistite da computer*.

Abstract The purpose of this note is twofold. First, we study some "curious" numbers, the well-known factorions, which are integers equal to the sum of the factorials of their digits in a given base, juxtaposing them with semifactorions, which we define as numbers equal to the sum of the semifactorials of their digits. We introduce a formal estimate of the corresponding numerosities based on the Lambert function W, and thus determine a bound on the number of their digits in the chosen base. This reduces the search to a finite set, and makes the problem computable, for example, using Python. We then show how this problem can represent an interesting paradigmatic case – possible in a classroom – of the "finite reduction" technique, which allows us to solve important problems (first and foremost, historically, the Four Color Theorem) by reducing them with theoretical procedures to a finite number of cases, which can then be examined by an automatic system: the so-called *computer-assisted proofs*.

1 Introduzione

Il termine *fattorione*[1] è il nome giocoso attribuito da Pickover (2005) ad ogni intero positivo uguale alla somma dei fattoriali delle sue cifre decimali: ne esistono solo 4, popolarmente noti, che sono: $1, 2, 145$ e $n = 40585$ (si ha $40585 = 4! + 0! + 5! + 8! + 5!$). L'estensione della derinizione ad una base $b \geq 2$ arbitraria è immediato. Qui, oltre ai fattorioni, introduciamo anche i *semifattorioni*. Formalmente:

Definizione *Sia $b \geq 2$ una base intera. Per un intero $n \geq 0$, indichiamo con $n = \sum_{i=0}^{k-1} d_i b^i$ il suo sviluppo in base b, con $0 \leq d_i < b$, e siano:*

$$S_1(n) = \sum_{i=0}^{k-1} d_i!, \quad S_2(n) = \sum_{i=0}^{k-1} d_i!!$$

Si dice che n è un fattorione *in base b se $S_1(n) = n$, e che è un* semifattorione[2] *in base b se $S_2(n) = n$.*

I fattorioni banali sono $1 = 1!$, in ogni base $b > 1$, e $2 = 2!$ in ogni base $b > 2$. I semifattorioni banali sono ancora $1 = 1!!$, $2 = 2!!$ e $3 = 3!!$, mentre lo zero non è mai un fattorione né un semifattorione, in quanto $0! = 0!! = 1$.

Dimostreremo che, quale che sia b, sia i fattorioni che i semifattorioni sono finiti, e ne invidueremo alcuni, per valori particolari di b, con un software computerizzato. La nota quindi può rappresentare un interessante caso, al contempo semplice, ma non banale, – proponibile a livello didattico – della tecnica di "riduzione al finito", che consente di risolvere importanti problemi (primo fra tutti, storicamente, il Teorema dei Quattro Colori) riportandoli con procedimenti teorici ad un numero finito di casi, che poi possono essere esaminati da un sistema automatico: le cosiddette *dimostrazioni assistite da computer*.

2 Riduzione al finito

Proposizione Ogni *fattorione (risp. semifattorione) n in base b verifica la limitazione*

$$n \leq \lfloor k^* \rfloor \cdot (b-1)! \qquad (\textit{risp. } n \leq \lfloor k^{**} \rfloor \cdot (b-1)!!)$$

dove $\lfloor \cdot \rfloor$ è la funzione "parte intera", e, detto W_{-1} il ramo inferiore della funzione W di Lambert:[3]

$$k^* = -\frac{1}{\log b} W_{-1}\left(-\frac{\log b}{b(b-1)!}\right), \qquad k^{**} = -\frac{1}{\log b} W_{-1}\left(-\frac{\log b}{b(b-1)!!}\right)$$

Dimostrazione Adattiamo argomentazioni classiche sulla limitazione del numero di cifre, trattando i due casi in parallelo. Semplifichiamo la trattazione definendo $M = (b-1)!$ e $S(n) = S_1(n)$ nel caso dei fattorioni, e analogamente $M = (b-1)!!$ e $S(n) = S_2(n)$ nel caso dei semifattorioni. Allora, se n ha k cifre (in base b), per la somma $S(n)$ si ha $kM \geq S(n)$, e banalmente $n \geq b^{k-1}$. Se $S(n) = n$ si deve avere $kM \geq S(n) = n \geq b^{k-1}$, per cui la condizione necessaria per l'esistenza di soluzioni con k cifre di $S(n) = n$, in entrambi i casi, è:

$$kM \geq b^{k-1}$$

[1] In inglese: *factorion*.

[2] Ricordiamo che i due punti esclamativi nella notazione $n!!$ del semifattoriale ricordando che si deve andare indietro a moltiplicare, a partire da n, a passo doppio, per cui, per $n!!$, si usa anche il termine *doppio fattoriale*. Qui preferiamo *semifattoriale*, termine che sottolinea che i termini da moltiplicare per il calcolo di $n!!$ sono circa la metà di quelli del fattoriale $n!$.

[3] Per la definizione della funzione W di Lambert si veda l'Appendice. Cf. anche `https://it.wikipedia.org/wiki/Funzione_W_di_Lambert`, oppure `https://mathworld.wolfram.com/LambertW-Function.html`

Per $b > 2$ (e quindi M) fissati, omesso il casi banale $k = 1$ (una sola cifra), come funzione di $k > 1$ la parte lineare kM supera l'esponenziale solo prima del loro punto di intersezione. Quindi, fissato $b > 2$, studiamo l'equazione trascendente $kM = b^{k-1}$ nell'incognita reale k. Usando $b^k = e^{k \log b}$ l'equazione diventa $kM = (1/b)\, e^{k \log b}$. Moltiplicando per $\log b$ e riordinando si ha $k \cdot \log b \cdot e^{-k \log b} = (\log b)/(Mb)$.

Ponendo $x = -k \log b$ otteniamo $xe^x = -(\log b)/(Mb)$ e quindi si ha:

$$x = W_{-1}\left(-\frac{\log b}{Mb}\right)$$

È bene accertarsi che questo abbia senso, ossia che i valori della successione $a_b = -(\log b)/(Mb)$ (per $b \geq 2$) siano tutti nel "campo di esistenza" della funzione. Per $t > 0$ reale, la funzione $t \longmapsto -(\log t)/t$ è decrescente nell'intervallo $(0, e]$ ed crescente per $t > e$. Tuttavia, i due valori "critici" $-\log(2)/2 \sim -0.35$ e $-\log(3)/3 \sim -0.37$ sono (largamente) maggiori di $-1/e \sim -1.28$ e dividendo per M restano *a fortiori* nell'intervallo $[-1/e, 0)$, ossia nel dominio del "ramo inferiore" W_{-1} della W di Lambert.[4] Ricordando che $x = -k \log b$, otteniamo la soluzione formale

$$k = -\frac{1}{\log b}\, W_{-1}\left(-\frac{\log b}{Mb}\right) \tag{1}$$

dove $k = k^*$ se $M = (b-1)!$ mentre $k = k^{**}$ se $M = (b-1)!!$. Quindi, affinché esistano fattorioni o semifattorioni, è necessario che il numero delle loro cifre non superi il k della (1), quindi ancora, trattandosi di un intero, può essere al massimo uguale alla parte intera $\lfloor k \rfloor$. Ricordando la condizione $n \geq kM$, abbiamo concluso. ■

La W è implementata in moltissimi software, ad esempio in *Wolfram Alpha* (Wolfram Alpha LLC), disponibile online, ma anche nel popolare *GeoGebra* (GeoGebra Team, 2025), largamente utilizzato nelle scuole. Questo consente di ottenere i valori numerici dei k^* o k^{**}reali[5] ad esempio per $b = 2, \ldots, 17$. In definitiva ogni fattorione n in base b verifica la limitazione

$$n \leq \lfloor k^* \rfloor \cdot (b-1)!$$

Come esempi nel caso dei fattorioni, per la base $b = 10$ si ha $k^* = 7.4308$, quindi in base 10 lo spazio di ricerca (finito) è quello degli interi $\leq 7 \cdot 9! = 2\,540\,160$ (~ 2.5 *milioni*); mentre in base $b = 13$ si ha $k^* = 9.67737$, e quindi, in base 13, lo spazio di ricerca è quello degli interi $\leq 9 \cdot 12! = 4\,311\,014\,400$ (~ 4.3 *miliardi*), che, seppur sempre finito, è circa 2700 volte più grande rispetto all'esempio precedente. In un laboratorio di matematica quindi, il caso dei fattorioni risulta rapidamente intrattabile in termini di tempo di calcolo,[6] mentre i semifattorioni appaiono più trattabili (a livello scolastico).

3 Implementazione in Python

In questa sezione viene mostrata l'implementazione in Python (PSF, 2026) dell'algoritmo descritto precedentemente per i semifattorioni. Lo script (copiabile) è un'adattamento di ChatGPT

[4] Poiché $W_0(a_b) \in [-1, 0)$, il valore $-\frac{1}{\log b} W_0(a_b)$ risulta sempre compreso tra 0 e $1/\log b$, e quindi è troppo piccolo per rappresentare il numero di cifre rilevante. Il ramo corretto è dunque sempre W_{-1}.

[5] Su *Wolfram Alpha*: `Table[N[-(1/Log[b])* ProductLog[-1, -Log[b]/(b*Factorial[b-1])]],{b,2,17}]` per i fattorioni, sostituendo `Factorial` con `Factorial2` per i semifattorioni.

[6] Ammettendo che il tempo di calcolo per i fattorioni in base $b = 10$ sia circa $1''$, per $b = 13$ ci si aspetta un tempo 2700 volte superiore, di circa $2700'' = 45'$. Questo dato è stato confermato su un iMac domestico con chip Apple M3, che ha prodotto i classici fattorioni decimali ben noti, ossia 1, 2, 145 e 40585 in $< 1''$, mentre per $b = 13$ ha richiesto esattamente $46'$ per calcolare i tre fattorioni 1, 2, 519326767.

(OpenAI, 2026) del codice disponibile al sito ufficiale `https://oeis.org/A014080` per i fattorioni. L'adattamento include una barra di avanzamento per il tempo di calcolo, trascorso e rimanente. Naturalmente, la stima di k nel programma può avvenire con un ciclo "while", senza bisogno di ricorrere alla formula chiusa. Presentiamo qui alcuni risultati e successivamente il codice usato.

Base	Semifattorioni
2	[1, 2]
3	[1, 2]
4	[1, 2, 3]
5	[1, 2, 3, 9]
6	[1, 2, 3, 17]
7	[1, 2, 3, 97]
8	[1, 2, 3, 49, 50, 51]
9	[1, 2, 3, 400]
10	[1, 2, 3, 107]
11	[1, 2, 3, 16]
12	[1, 2, 3, 1053]
13	[1, 2, 3]
14	[1, 2, 3, 1891, 1892, 1893, 191666]
15	[1, 2, 3, 51, 96, 106, 107, 108, 181603]
16	[1, 2, 3, 2083607]
17	[1, 2, 3, 2729566, 4101078]
18	[1, 2, 3]
19	[1, 2, 3, 4189572]

3.1 Esempio

Verifichiamo – per controllo e come proposta di "esercizio" – che $n = 4189572$ è un semifattorione in base $b = 19$. Convertendo la base da 10 a 19 si ha $n = 1d2f8f_{19}$. Valutando le cifre aggiuntive $d = 13$ e $f = 15$ si conferma il risultato: $2!! + 15!! + 1!! + 12!! + 9!! + 15!! = 1 + 135135 + 2 + 2027025 + 384 + 2027025 = 4189572$.

3.2 Una questione strutturale

A parte la terna banale $(1, 2, 3)$ che si presenta sempre, si notano occorrenze di altre terne di numeri consecutivi, come $(49, 50, 51)$ in base 8, $(1891, 1892, 1893)$ in base 14 e $(106, 107, 108)$ in base 15. Le genesi sono diverse. Valgono questi risultati:

A. Se $m \geq 4$, e se $b = m!!/m > m$, allora esiste in base b una terna di semifattorioni consecutivi $n = mb + k = m!! + k!!$ $(k = 1, 2, 3)$. Infatti la rappresentazione di n in base b è esattamente $(mk)_b$. In questo caso rientrano la base $8 = 6!!/6 > 6$, che genera la terna $6 \cdot 8 + k$ $(k = 1, 2, 3)$, ossia $(49, 50, 51)$; e la base $15 = 7!!/7 > 7$, che genera la terna $7 \cdot 15 + k$ $(k = 1, 2, 3)$, ossia $(106, 107, 108)$.

B. Si possono così generare esercizi in stile "quasi–olimpico", come ad esempio: *provare che* $(385, 386, 387)$ *è una terna di semifattorioni in base* $b = 48$. In base 48, la terna si scrive $(81_{48}, 82_{48}, 83_{48})$. Con $m = 8$ vale la $b = m!!/m > m$, per cui, ancora: $mb + k = m!! + k!!$ $(k = 1, 2, 3)$.

C. La terna della base 14 non rientra nello schema precedente. Viene dalla formula $2m!! = mb^2 + mb$ che vale in quel caso con $b = 14$ e $m = 9$; infatti $2 \cdot 9!! = 1890 = 9 \cdot 14^2 + 9 \cdot 14$; se vale la formula, in base b abbiamo un numero a tre cifre $n = (m\,m\,k)_b = 2m!! + k$, e siccome $k!! = k$, $(k = 1, 2, 3)$ otteniamo $2m!! + k = m!! + m!! + k!!$.

D. Questa breve analisi porta a pensare che la genesi dei semifattorioni (come quella dei fattorioni) dipenda da un mix di struttura e di casualità, che potrebbe essere oggetto di indagine, almeno al livello della matematica "curiosa e dilettevole".

3.3 Codice copiabile

```
from tqdm import tqdm

def digits_base(n,b):
    while n: n,r=divmod(n,b); yield r

def dfact(n):
    if n<=1:return 1
    p=1
    while n>1:p*=n;n-=2
    return p

def upper_bound(b,cache):
    m,k=cache[b-1],1
    while k*m>=b**(k-1):k+=1
    return (k-1)*m

def semi_factorions_base(b):
    cache=[dfact(i) for i in range(b)]
    U=upper_bound(b,cache)
    R=[]
    for n in tqdm(range(1,U+1),desc=f"Base {b}"):
        s=0
        for d in digits_base(n,b):
            s+=cache[d]
            if s>n:break
        if s==n:R.append(n)
    return R

if __name__=="__main__":
    print(semi_factorions_base(int(input("Base b: "))))
```

4 Appendice

La **funzione di Lambert** $W(z)$ è l'inversa multivoca di $f(w) = we^w$, cioè risolve l'equazione $we^w = z$. Il grafico è costituito da due rami reali principali: il *ramo superiore* $W_0(x)$ (per $x \geq -1/e$) ed il *ramo inferiore* $W_{-1}(x)$ (per $-1/e \leq x < 0$), che si incontrano nel punto di diramazione $(-1/e, -1)$. Si veda la Figura 1. La funzione W è un esempio interessante di una funzione "antica" (Eulero, XVIII sec.) che è diventata standard nei software solo negli anni '80–'90. Il primo sistema importante che implementò la funzione W come oggetto esplicito fu *Maple*. Dopo questa implementazione, Robert Corless, David Jeffrey ed altri (con la partecipazione importante di Donald Knuth) pubblicarono il celebre lavoro del 1996 (Corless *et al.*, 1996) che rese la funzione "standard" in matematica applicata.[7]

Da lì iniziò la diffusione nei CAS, ed attualmente è implementata anche in *GeoGebra* (GeoGebra Team, 2025). Notiamo ancora che nel numero di febbraio 2000 di FOCUS, la newsletter della Mathematical Association of America, comparve un articolo in cui si proponeva la funzione W come valida candidata a entrare tra le nuove funzioni elementari da introdurre nell'insegnamento scolastico e nei libri di testo (Gouvêa, 2000). Questa nota può proporre uno stimolo in questa direzione.

[7] Il lavoro fu completato nel gennaio del 1993, cf.: `https://cs.uwaterloo.ca/research/tr/1993/03/W.pdf`.

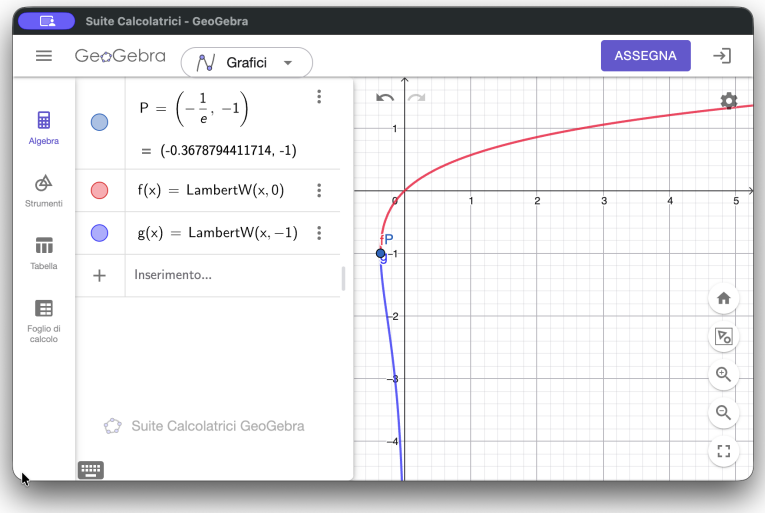

Figura 1: Il grafico della funzione W di Lambert (multivoca) come tracciato da *GeoGebra*; in rosso il ramo superiore W_0, in blu quello inferiore W_{-1}; è inoltre evidenziato il punto $(-1/e, -1)$ di biforcazione.

Bibliografia

Corless, R. M., Gonnet, G. H., Hare, D. E. G., Jeffrey, D. J., & Knuth, D. E. (1996). On the Lambert W function. Advances in Computational Mathematics, 5(1), 329–359. `https://doi.org/10.1007/BF02124750`.

GeoGebra Team. (2025). GeoGebra (Version 6.0.920) [Computer software]. `https://www.geogebra.org`.

Gouvêa, F. (Ed.). (2000). Time for a new elementary function? FOCUS: Newsletter of the Mathematical Association of America, 20, 2.

OpenAI. (2026). ChatGPT (versione 4o) [Modello linguistico di grandi dimensioni]. `chat.openai.com`.

Pickover, C. A. (2005). A passion for mathematics: Numbers, puzzles, madness, religion, and the quest for reality. John Wiley & Sons.

Python Software Foundation. (2026). Python (Versione 3.14) [Software computerizzato]. `https://www.python.org/`.

Wolfram Alpha LLC. (n.d.). Wolfram—Alpha: Computational Intelligence. Consultato il 21 Marzo 2026, da `https://www.wolframalpha.com`.

L'autore Sergio Invernizzi, già ordinario di *Analisi matematica* e di *Matematiche complementari* nell'Università di Trieste, dopo il ritiro dall'attività accademica ufficiale (dicembre 2013) dedica parte del tempo "libero" alla redazione di brevi note ed articoli non–specialistici di matematica. Ricordando una citazione di G. H. Hardy: "Young men should prove theorems, old men should write books".

Padriciano, Trieste, 20 marzo 2024, 14:32